Technology Stack Justification

เหตุผลการเลือกใช้เทคโนโลยีในโครงการ
Hybrid Cloud Infrastructure Monitoring Dashboard

เอกสารหมายเลข 1.2 — Phase 1: Design & Setup

โครงงานวิศวกรรมคอมพิวเตอร์ ปีสุดท้าย
Framework: Zabbix 7.x | Cloud: AWS + GCP Free Tier

สารบัญ

1. บทนำและหลักเกณฑ์การเลือกเทคโนโลยี
2. Zabbix — ระบบ Monitoring หลัก
3. MariaDB — ฐานข้อมูล
4. Grafana — ระบบ Visualization
5. Python — Cost Aggregator
6. Zabbix Agent 2 — ตัวเก็บข้อมูล
7. SNMP v2/v3 — Network Monitoring
8. Ansible — Infrastructure as Code
9. LINE Notify — ระบบแจ้งเตือน
10. AWS EC2 + GCP e2-micro — Cloud Compute
11. Ubuntu 22.04 LTS — ระบบปฏิบัติการ
12. ตารางสรุปภาพรวม Technology Stack

1. บทนำและหลักเกณฑ์การเลือกเทคโนโลยี

เอกสารฉบับนี้อธิบายเหตุผลเบื้องหลังการเลือกใช้เครื่องมือและเทคโนโลยีแต่ละตัวในโครงการ Hybrid Cloud Infrastructure Monitoring Dashboard โดยเปรียบเทียบกับทางเลือกอื่นที่มีอยู่ในตลาด เพื่อแสดงให้เห็นว่าการตัดสินใจแต่ละครั้งผ่านการพิจารณาข้อดี ข้อจำกัด และบริบทของโครงการอย่างรอบด้าน ไม่ใช่การเลือกตามความคุ้นเคยเพียงอย่างเดียว

เกณฑ์หลักที่ใช้พิจารณาเทคโนโลยีแต่ละตัวมี 5 ข้อ ได้แก่

- **ต้นทุน** — ต้องเป็น Open-source หรืออยู่ในระดับ Free Tier เนื่องจากโครงการมีงบประมาณจำกัดที่ 0 บาท
- **ความสามารถรองรับ Hybrid Environment** — ต้องจัดการได้ทั้ง On-premise และ Multi-cloud ในระบบเดียว
- **ความสามารถขยายระบบ (Scalability)** — ต้องรองรับการเพิ่ม Host และ VM ผ่าน Proxy/Agent ได้โดยไม่กระทบ Performance
- **ความพร้อมของเอกสารและ Community** — เพื่อลดความเสี่ยงเวลาที่นักศึกษาต้องแก้ปัญหาเอง
- **ความสอดคล้องกับวัตถุประสงค์การเรียนรู้** — ต้องช่วยให้ได้ฝึกทักษะที่ตรงกับสายงาน DevOps/Infrastructure จริง

---

2. Zabbix 7.x — ระบบ Monitoring หลัก

เทียบกับ:

ทางเลือก	เหตุผลที่ไม่เลือกเป็นตัวหลัก
Prometheus + Grafana	เป็น Pull-based (scrape) ที่ออกแบบมาสำหรับ Cloud-native/Container เป็นหลัก การ Monitor อุปกรณ์ On-prem แบบดั้งเดิม เช่น Switch/Router ผ่าน SNMP ทำได้ไม่สะดวกเท่า Zabbix ต้องพึ่ง Exporter เพิ่มเติมหลายตัว
Nagios / Icinga	UI และการจัดการ Configuration ยังเป็นแบบเก่า ไม่มี Auto-discovery ที่ยืดหยุ่นเท่า Zabbix Low-Level Discovery (LLD) และ Ecosystem สำหรับ Cloud API ไม่กว้างเท่า
Datadog / New Relic	เป็น SaaS เชิงพาณิชย์ มีค่าใช้จ่ายตาม Host/Metric ซึ่งขัดกับงบประมาณ 0 บาทของโครงการ และไม่เหมาะกับการสาธิตในบริบทการศึกษาที่ต้องโฮสต์เองเพื่อแสดงความเข้าใจ Infrastructure

เหตุผลที่เลือก Zabbix

- **Open-source ทั้งระบบ** ไม่มีค่าใช้จ่าย ตรงกับข้อจำกัดด้านงบประมาณของโครงการ
- **รองรับสถาปัตยกรรมแบบ Proxy โดยตรง** — Zabbix Proxy ถูกออกแบบมาเพื่อกระจายการเก็บข้อมูลจากหลาย Network Segment (On-prem, AWS VPC, GCP VPC) แล้วส่งกลับมารวมที่ Server ศูนย์กลาง ตรงกับโจทย์ Hybrid Cloud ของโครงการนี้พอดี
- **รองรับ SNMP แบบ Native** เหมาะกับการ Monitor อุปกรณ์เครือข่ายในแล็บที่เป็น Passive Device ซึ่งไม่มี Agent ให้ติดตั้ง
- **มี Low-Level Discovery (LLD)** ทำให้ Auto-discovery VM ใหม่บน AWS/GCP ได้โดยไม่ต้อง Config มือทุกเครื่อง ซึ่งเป็นหนึ่งในเกณฑ์ประเมินของโครงการ (น้ำหนัก 10%)
- **รองรับ Template และ External Check** ทำให้สามารถผลัก Cost Metric จาก Python Script เข้ามาเป็น Custom Item ได้ง่าย ผ่าน Zabbix Sender Protocol ซึ่งเป็นหัวใจของ Cost Visibility Layer
- **REST API ครบถ้วน** ทำให้ Grafana ดึงข้อมูลไปแสดงผลต่อได้สะดวก

> **ข้อจำกัดที่ต้องยอมรับ:** Zabbix Native UI ไม่สวยงามและปรับแต่งได้จำกัดเมื่อเทียบกับ Grafana จึงต้องใช้ Grafana เป็นชั้น Visualization แยกต่างหาก (ดูหัวข้อที่ 4) และการตั้งค่าเริ่มต้นมีความซับซ้อนกว่า Prometheus เล็กน้อยในส่วนของ Web Frontend ที่ต้องพึ่ง PHP

---

3. MariaDB — ฐานข้อมูลสำหรับ Zabbix

เทียบกับ:

ทางเลือก	เหตุผลที่ไม่เลือก
PostgreSQL	Zabbix รองรับทั้งคู่เท่ากัน แต่ MariaDB เป็นตัวเลือกที่ Community ของ Zabbix ใช้งานมากที่สุดและมีตัวอย่าง Config/Tuning สำหรับ Zabbix โดยเฉพาะมากกว่า ลดความเสี่ยงเวลาต้อง Debug Performance
MySQL (Oracle)	MariaDB เป็น Fork ที่ยังคง Compatible กับ MySQL แต่เป็น Community-driven License เต็มรูปแบบ ไม่มีความเสี่ยงเรื่องการเปลี่ยนแปลง License ในอนาคต

เหตุผลที่เลือก MariaDB

- เป็นฐานข้อมูลที่ Zabbix แนะนำอย่างเป็นทางการ และมี Partitioning support ที่ช่วยจัดการข้อมูล History/Trend ปริมาณมากจากหลาย Environment ได้มีประสิทธิภาพ
- Resource Footprint เบากว่า PostgreSQL เล็กน้อยเมื่อรันบน Lab Machine ที่มีสเปกจำกัด (2 CPU / 4GB RAM)
- Open-source 100% ไม่มีค่าใช้จ่าย

4. Grafana 10.x — ระบบ Visualization

เทียบกับ:

ทางเลือก	เหตุผลที่ไม่เลือก
Zabbix Native Dashboard	ทำ Dashboard ได้ แต่ปรับแต่ง Layout/Panel Type ได้จำกัด และไม่สามารถรวมข้อมูลจากหลาย Data Source (เช่น InfluxDB สำหรับ Cost Trend) เข้ามาแสดงในหน้าเดียวกันได้ดีเท่า Grafana
Kibana	ออกแบบมาคู่กับ Elasticsearch เป็นหลัก ถ้าเลือกใช้ต้องเพิ่ม ELK Stack ทั้งชุดเข้ามาในระบบ ซึ่งเกินความจำเป็นของโครงการและเพิ่มภาระ Resource ให้ Lab Machine

เหตุผลที่เลือก Grafana

- **รวม Data Source ได้หลากหลายในหน้าเดียว** — เชื่อมต่อกับ Zabbix ผ่าน Official Plugin ได้โดยตรง และยังเชื่อมกับ InfluxDB (สำหรับเก็บ Cost Trend ระยะยาว) ได้พร้อมกัน ทำให้ Dashboard เดียวแสดงทั้ง Performance และ Cost ได้ตามโจทย์หลักของโครงการ
- **Panel Type หลากหลายและปรับแต่งง่าย** เหมาะกับการทำ Cost vs. CPU Utilization Efficiency Score ซึ่งเป็นจุดขายของโครงการ
- **Alerting ในตัว** สามารถตั้งแจ้งเตือนเมื่อ Cost เกิน Budget ได้โดยไม่ต้องพึ่ง Zabbix Trigger เพียงอย่างเดียว
- **Free และ Self-hosted ได้เต็มรูปแบบ** ตรงกับข้อจำกัดด้านงบประมาณ

5. Python — Cost Aggregator Script

เทียบกับ:

ทางเลือก	เหตุผลที่ไม่เลือก
Bash Script	เขียนเรียก REST API และ Parse JSON ที่ซับซ้อนได้ยากกว่า Python มาก ไม่เหมาะกับ Logic ที่ต้องคำนวณ Cost-per-instance และจัดการ Error จาก API หลายตัว
Go	Performance ดีกว่า แต่ใช้เวลาพัฒนานานกว่า และ SDK/Library สำหรับ AWS Cost Explorer, GCP Billing และ Zabbix Sender ในฝั่ง Python สมบูรณ์และมีตัวอย่างการใช้งานมากกว่า
Node.js	ทำได้เช่นกัน แต่ทีมงาน/นักศึกษามีความคุ้นเคยกับ Python มากกว่าในบริบทงาน Infrastructure/DevOps และ Python เป็นภาษาที่ Ansible ก็ใช้อยู่แล้ว ทำให้ Stack โดยรวมเรียบง่ายขึ้น

เหตุผลที่เลือก Python

- **มี Library รองรับ Cloud API โดยตรงและครบถ้วน** — `boto3` สำหรับเรียก AWS Cost Explorer API และ `google-cloud-billing` สำหรับ GCP Billing API ทำให้ไม่ต้องเขียน HTTP Request และจัดการ Authentication (OAuth/IAM) เอง ลดความซับซ้อนและข้อผิดพลาดได้มาก
- **มี Library `py-zabbix` หรือเรียก Zabbix Sender Protocol ได้ตรง** ทำให้ส่ง Cost Metric เข้า Zabbix เป็น External Item ได้สะดวก
- **Ecosystem Data Processing แข็งแรง** — ใช้ `pandas` ช่วยคำนวณ Cost-per-instance และ Efficiency Score ก่อนส่งเข้า Zabbix ได้ง่าย
- **เขียน Cron Job/Scheduler ได้ตรงไปตรงมา** เหมาะกับงาน Poll Billing API แบบรายวันตามที่ออกแบบไว้

---

6. Zabbix Agent 2 — ตัวเก็บข้อมูลบน Host

เหตุผลที่เลือก (เทียบกับ Zabbix Agent รุ่นเดิม)

- เขียนด้วยภาษา Go ทำให้ใช้ Resource น้อยกว่า Agent รุ่นเดิมที่เขียนด้วย C แบบ Thread-per-check เมื่อต้อง Monitor หลาย Item พร้อมกันบน VM ขนาดเล็กอย่าง `t2.micro` และ `e2-micro`
- รองรับระบบ Plugin ในตัว ทำให้ Monitor Docker Container ได้โดยไม่ต้องติดตั้ง Exporter แยก ตรงกับ Feature ที่ต้องการในหัวข้อ Docker Monitoring
- Backward-compatible กับ Agent รุ่นเดิม ทำให้ Config และ Template เดิมยังใช้ได้

---

7. SNMP v2/v3 — Network Device Monitoring

เหตุผลที่เลือก

- อุปกรณ์เครือข่าย เช่น Switch/Router ในแล็บส่วนใหญ่ไม่รองรับการติดตั้ง Agent จึงจำเป็นต้องใช้ Protocol มาตรฐานอุตสาหกรรมอย่าง SNMP ซึ่งอุปกรณ์เกือบทุกยี่ห้อรองรับ
- เลือกใช้ **SNMP v3** เป็นหลักเมื่อทำได้ เนื่องจากมี Authentication และ Encryption ในตัว ปลอดภัยกว่า v2c ที่ส่ง Community String แบบ Plain Text ส่วน v2c ใช้เป็น Fallback สำหรับอุปกรณ์รุ่นเก่าที่ไม่รองรับ v3 เท่านั้น

---

8. Ansible — Infrastructure as Code

เทียบกับ:

ทางเลือก	เหตุผลที่ไม่เลือกเป็นตัวหลัก
Terraform เพียงอย่างเดียว	Terraform เหมาะกับการสร้าง/จัดการ Infrastructure ระดับ Cloud Resource (VPC, EC2, VM) แต่ไม่เหมาะกับการเข้าไป Config ภายในเครื่อง เช่น ติดตั้ง Zabbix Agent หรือแก้ไฟล์ Config จึงใช้ทั้งสองตัวร่วมกันตามความถนัด: Terraform สร้างทรัพยากร Cloud, Ansible Config ภายในเครื่อง
Shell Script ล้วน	ไม่มีแนวคิดเรื่อง Idempotency (รันซ้ำแล้วผลลัพธ์เหมือนเดิม) ทำให้เสี่ยงต่อการเกิด Configuration Drift เมื่อต้อง Deploy ซ้ำหลายรอบระหว่างพัฒนา

เหตุผลที่เลือก Ansible

- **Agentless** — ใช้ SSH ในการเชื่อมต่อ ไม่ต้องติดตั้งซอฟต์แวร์เพิ่มบน Target Host ลดความซับซ้อนเวลา Deploy Zabbix Proxy ไปยัง AWS/GCP VPC
- เขียน Playbook เป็น YAML อ่านง่าย เหมาะกับการทำเอกสารประกอบ (Work Product 3.1) ให้คนอื่นเข้าใจและรันตามได้
- Idempotent — รัน Playbook ซ้ำได้โดยไม่สร้างปัญหา ตรงกับเกณฑ์ประเมิน "Reproducible setup via Ansible/Terraform" ในหัวข้อ Documentation & IaC

---

9. LINE Notify — ระบบแจ้งเตือน

เทียบกับ:

ทางเลือก	เหตุผลที่ไม่เลือกเป็นตัวหลัก
Slack	ทีมงาน/ผู้ดูแลระบบในบริบทประเทศไทยส่วนใหญ่ใช้ LINE เป็นช่องทางหลักในชีวิตประจำวันมากกว่า Slack ทำให้การแจ้งเตือนถูกมองเห็นและตอบสนองได้เร็วกว่าในสถานการณ์จริง
Email เพียงอย่างเดียว	การแจ้งเตือนผ่าน Email มักมี Latency สูงกว่าและถูกอ่านช้ากว่าการแจ้งเตือนแบบ Push Notification บนมือถือ จึงใช้ Email เป็นช่องทางสำรอง (Escalation) ไม่ใช่ช่องทางหลัก

เหตุผลที่เลือก LINE Notify

- ตั้งค่าง่าย ใช้ Webhook/Token เรียกผ่าน HTTP POST ได้ตรงจาก Zabbix Media Type แบบ Custom Script
- ฟรี ไม่มีค่าใช้จ่าย และ Latency ต่ำ เหมาะกับ Use Case ที่ต้องการแจ้งเตือนเร่งด่วน เช่น Host Down หรือ Cost เกิน Budget

---

10. AWS EC2 (t2.micro) + GCP e2-micro — Cloud Compute

เหตุผลที่เลือกใช้ทั้งสอง Cloud พร้อมกัน

- โจทย์ของโครงการคือ "Hybrid **และ Multi-cloud**" การใช้ทั้ง AWS และ GCP พร้อมกันแสดงให้เห็นว่าการออกแบบ Proxy Layer ของ Zabbix รองรับ Provider ที่หลากหลายจริง ไม่ได้ผูกกับ API ของเจ้าใดเจ้าหนึ่ง
- **AWS EC2 t2.micro** อยู่ใน Free Tier 12 เดือนแรก และมี Cost Explorer API ที่ให้ข้อมูลละเอียดระดับ Resource Tag ซึ่งจำเป็นต่อการทำ Cost-per-instance
- **GCP e2-micro** อยู่ใน Always Free Tier (ไม่มีวันหมดอายุ ตราบเท่าที่อยู่ใน Region ที่กำหนด) ทำให้ประหยัดค่าใช้จ่ายระยะยาวกว่า และมี Billing Export ไปยัง BigQuery ที่ให้รายละเอียด Cost ที่ดีเช่นกัน

> **สรุป:** การใช้ทั้งสอง Provider ทำให้ระบบ Auto-discovery และ Cost Aggregator ต้องถูกออกแบบให้ Provider-agnostic มากขึ้น ซึ่งเป็นทักษะสำคัญที่ตรงกับ Learning Outcome ของโครงการ

---

11. Ubuntu 22.04 LTS — ระบบปฏิบัติการ

เหตุผลที่เลือก

- เป็น LTS (Long-Term Support) ได้รับ Security Patch ต่อเนื่องตลอดระยะเวลาโครงการ (16 สัปดาห์) โดยไม่ต้อง Upgrade กลางทาง
- Zabbix, Grafana และ Docker มี Official Package/Repository รองรับ Ubuntu โดยตรง ลดปัญหาความเข้ากันไม่ได้ของ Dependency
- เป็น OS ที่ทั้ง AWS AMI และ GCP Image รองรับเป็นมาตรฐาน ทำให้ Config เครื่อง On-prem และ Cloud VM มีพื้นฐานเดียวกัน ลดความซับซ้อนของ Ansible Playbook

---

12. ตารางสรุปภาพรวม Technology Stack

ส่วนประกอบ	เทคโนโลยีที่เลือก	เหตุผลสั้น ๆ
Monitoring Core	Zabbix 7.x	Open-source, รองรับ Proxy + SNMP + LLD ครบสำหรับ Hybrid Cloud
ฐานข้อมูล	MariaDB	ตัวเลือกที่ Zabbix แนะนำ, Community ใหญ่, Resource เบา
Visualization	Grafana 10.x	รวมหลาย Data Source ได้ ทำ Dashboard สวยและปรับแต่งได้เต็มที่
Cost Script	Python 3 (boto3, google-cloud-billing)	มี Library เรียก Cloud Billing API พร้อมใช้ ลดงานพัฒนา
Agent	Zabbix Agent 2	เบา เขียนด้วย Go, รองรับ Plugin สำหรับ Docker
Network Monitoring	SNMP v2/v3	มาตรฐานอุตสาหกรรม ใช้กับอุปกรณ์ที่ไม่มี Agent
Automation	Ansible	Agentless, Idempotent, เอกสารอ่านง่าย
Alerting	LINE Notify + Email	แจ้งเตือนไว เหมาะกับผู้ใช้ในไทย, Email เป็น Backup
Cloud Compute	AWS EC2 t2.micro + GCP e2-micro	Free Tier ทั้งคู่, ครอบคลุมทั้ง Multi-cloud ตามโจทย์
OS	Ubuntu 22.04 LTS	เสถียร, รองรับ Package ทุกตัวในระบบโดยตรง

บทสรุปโดยรวม: ทุกเทคโนโลยีที่เลือกในโครงการนี้ผ่านเกณฑ์ 5 ข้อที่กำหนดไว้ในหัวข้อที่ 1 อย่างครบถ้วน คือ ไม่มีค่าใช้จ่าย รองรับ Hybrid/Multi-cloud ได้จริง ขยายระบบได้ในอนาคต มีเอกสาร/Community รองรับเพียงพอ

> สำหรับนักศึกษาที่ทำงานคนเดียว และช่วยฝึกทักษะที่ตรงกับสายงาน Infrastructure/DevOps ในตลาดแรงงานจริง